

Functions

CSCI 1101

2024-01-26

Writing Script Files

Last time we used python as a fancy calculator by evaluating numeric expressions in the **interactive interpreter**.

Clearly, this way of programming is not very scalable.

We can use a **script file** to save our programs and run them whenever we want.

Ask your program editor to create a new script file from the `File` menu.

In the new file type the following assignment statement:

```
pi = 22 / 7
```

Save the file somewhere you can find it with a name that ends in “.py”.

Running Script Files

Ask python to evaluate the expression `pi`.
You should get an error.

Then ask python to read in the contents of the script file:

Idle: Run -> Run Module

Wing101: Source -> Evaluate File in Python Shell (or “play” button)

Now ask python to evaluate `pi` again.
This time python should know what you mean.

Defining Functions

Back in your script file enter the following:

```
def one_plus(num) :  
    return 1 + num
```

This is a **function definition**, a reusable bit of python code.

Anatomy of a Function Definition

Python functions have the following structure,

header: a declaration consisting of:

- the keyword “def”
- the **function name** (same naming rules as for variables)
- a comma-separated list of **function parameters** between parentheses
- the colon symbol “:”

body: any valid sequence of python statements that:

- is indented by one level from the header
- ends (at least implicitly) with a **return statement**

Function Input and Output

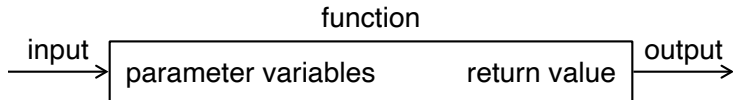
Parameter variables are the variables that occur in the function header.

They represent the inputs to a function.

Return value is the value of the expression in a return statement.

It represents the output of a function.

We can visualize this flow of information as follows:



Activity: Defining Functions

Recall the function that adds one to its input,

```
def one_plus(num) :  
    return 1 + num
```

Define a function called `two_times` that doubles its input.

Define a function called `twice_sum` that takes two numbers as input and returns the number that is twice their sum.

Calling Functions

In order to provide a function with its input and ask it to produce its output we **call** the function.

We do this using the following syntax:

```
<function_name>(<function_arguments>)
```

The **function arguments** can be any expressions of the same types as the corresponding parameters.

for example:

```
one_plus(2)
```


Activity: Calling Functions

Try calling the `one_plus`, `two_times` and `twice_sum` functions to return the following values without using explicit addition or multiplication:

- 2×3
- $1 + (2 \times 3)$
- $2 \times (2 + 3)$
- $2 \times (2 \times 3)$

Function Call Execution (simplified version)

Calling a function creates a detour in the flow of program execution:

1. The function argument expressions are evaluated.
2. Their values are assigned to the function parameter variables.
3. Control passes to the beginning of the function body.
4. Control flows within the function body until a return statement is reached.
5. The expression following the keyword `return` is evaluated to a return value.
6. The function call is abandoned and its variables are forgotten.
7. The return value becomes the value of the function call.
8. Control returns to place where the function was called.

Activity: Tracing Function Calls

Work out what python does for the following function calls:

```
one_plus(2 + 2)
```

```
two_times(one_plus(2 + 2))
```

```
one_plus(one_plus(2 + 2))
```

Function Specifications

As our functions grow more complicated it becomes important to clearly understand and communicate how they work.

We do this using a **function specification** written using a special comment at the beginning of a function definition called a **documentation string** (or “docstring”).

This is written using the following syntax:

```
def <function_name>(<function_parameters>) :  
    """  
    signature: <function_signature>  
    <function_description>  
    """  
    <function_body>
```

Function Specifications Ctd.

A **function signature** describes the number and types of the arguments and the type of the return value.

It is written according to the following syntax:

`(<types_of_arguments>) --> <type_of_result>`

For example, a function that takes one `int` argument and returns an `int` result has signature:

`(int) --> int`

A **function description** is ordinary English text describing the function's behavior.

Example: Function Specification

Here is our `one_plus` function with a complete docstring:

```
def one_plus(num) :  
    """  
    signature: (int) --> int  
    Returns the number one greater than its argument.  
    """  
    return 1 + num
```

This is overkill for such a simple function, but soon we will learn how to write much more complicated programs.

Activity: Function Specification

Add a specification to the `two_times` and `twice_sum` functions.

Notes on Style

`CodeRunner` can be opinionated about programming style.

Be sure to read its style rules and use the `PyLint` tool to check that you satisfy them.

Activity: PyLint Checking

Put your `two_times` and `twice_sum` functions through the *PyLint function checker* and fix any issues that it raises.

To Do:

Finish *Homework 1: Expressions, Variables, and Functions* on CodeRunner by 11:00AM next Thursday.